

7. TUTORIAL RECORD (Chapters)

CHAPTER 1

Objective:

Write A Prolog Program To Find The Sum Of Elements In Given List.

Requirements:

Processors: Intel Atom® processor or Intel® Core™ i3 processor., Disk space: 1 GB. ,
Operating systems: Windows* 7 or later, macOS, and Linux. Software Prolog

Theory:

We Will Define A Clause, List_Sum(List, Sum), This Will Return The Sum Of The Elements Of The List.

1. If The List Is Empty, Then Sum Will Be 0.
2. Represent List As [Head|Tail], Find Sum Of Tail Recursively And Store Them Into Sumtemp, Then Set Sum = Head + Sumtemp

Program:

```
List_Sum([],0).  
List_Sum([Head|Tail], Sum) :-  
    List_Sum(Tail,Sumtemp),  
    Sum Is Head + Sumtemp.
```

Output:

```
Yes  
| ?- List_Sum([5,12,69,112,48,4],Sum).
```

Sum = 250

```
Yes  
| ?- List_Sum([8,5,3,4,7,9,6,1],Sum).
```

Sum = 43

```
Yes  
| ?-
```

CHAPTER 2

Objective:

Write A Prolog Program To Delete An Element From Given List.

Requirements:

Processors: Intel Atom® processor or Intel® Core™ i3 processor., Disk space: 1 GB. ,

Operating systems: Windows* 7 or later, macOS, and Linux. Software Prolog.

Theory:

Suppose We Have A List L And An Element X, We Have To Delete X From L. So There Are Three Cases –

1. If X Is The Only Element, Then After Deleting It, It Will Return Empty List.
2. If X Is Head Of L, The Resultant List Will Be The Tail Part.
3. If X Is Present In The Tail Part, Then Delete From There Recursively.

Program:

```
List_Delete(X, [X], []).
```

```
List_Delete(X,[X|L1], L1).
```

```
List_Delete(X, [Y|L2], [Y|L1]) :- List_Delete(X,L2,L1).
```

Output:

```
| ?- List_Delete(A,[A,E,I,O,U],Newlist).
```

```
Newlist = [E,I,O,U] ?
```

Yes

```
| ?- List_Delete(A,[A],Newlist).
```

```
Newlist = [] ?
```

Yes

```
| ?- List_Delete(X,[A,E,I,O,U],[A,E,O,U]).
```

```
X = I ? ;
```

No

```
| ?-
```

CHAPTER 3

Objective:

Write A Prolog Code To Find The Last Element Of A Given Input List

Requirements:

Processors: Intel Atom® processor or Intel® Core™ i3 processor., Disk space: 1 GB. ,

Operating systems: Windows* 7 or later, macOS, and Linux. Software Prolog.

Theory:

Define A Predicate Last_Element(X, L) That Is True If X Is The Last Element Of List L

Program:

```
Last_Element(X, [X]).           % Base Case: X Is The Last Element Of A Singleton List
Last_Element(X, [_|T]) :- Last_Element(X, T).      % Recursive Case: X Is The Last
Element Of The Tail Of The List
```

Output:

```
Last_Element(X, [1, 2, 3, 4, 5]).
```

Which Should Return X = 5.

CHAPTER 4

Objective:

Write A Prolog Program To Check If Given Element Is A Member Of A Given List.

Requirements:

Processors: Intel Atom® processor or Intel® Core™ i3 processor., Disk space: 1 GB. ,

Operating systems: Windows* 7 or later, macOS, and Linux. Software Prolog.

Theory:

During This Operation, We Can Check Whether A Member X Is Present In List L Or Not? So How To Check This? Well, We Have To Define One Predicate To Do So. Suppose The Predicate Name Is List_Member(X,L). The Goal Of This Predicate Is To Check Whether X Is Present In L Or Not.

To Design This Predicate, We Can Follow These Observations. X Is A Member Of L If Either –

1. X Is Head Of L, Or
2. X Is A Member Of The Tail Of L

Program:

List_Member(X,[X|_]).

List_Member(X,[_|Tail]) :- List_Member(X,Tail).

Output:

| ?- List_Member(B,[A,B,C]).

True ?

Yes

| ?- List_Member(B,[A,[B,C]]).

No

| ?- List_Member([B,C],[A,[B,C]]).

True ?

Yes

| ?- List_Member(D,[A,B,C]).

No

| ?- List_Member(D,[A,B,C]).

CHAPTER 5

Objective:

Write A Prolog Program To Append List L1 To List L2 And Bind The Result To List L3.

Requirements:

Processors: Intel Atom® processor or Intel® Core™ i3 processor., Disk space: 1 GB. ,

Operating systems: Windows* 7 or later, macOS, and Linux. Software Prolog.

Theory:

Appending Two Lists Means Adding Two Lists Together, Or Adding One List As An Item. Now If The Item Is Present In The List, Then The Append Function Will Not Work. So We Will Create One Predicate Namely, List_Append(L1, L2, L3). The Following Are Some Observations –

1. Let A Is An Element, L1 Is A List, The Output Will Be L1 Also, When L1 Has A Already.
2. Otherwise New List Will Be $L2 = [A|L1]$.

In This Case, We Have Used (!) Symbol, That Is Known As Cut. So When The First Line Is Executed Successfully, Then We Cut It, So It Will Not Execute The Next Operation.

Program:

```
List_Member(X,[X|_]).
```

```
List_Member(X,[_|Tail]) :- List_Member(X,Tail).
```

```
List_Append(A,T,T) :- List_Member(A,T),!.
```

```
List_Append(A,T,[A|T]).
```

Output:

```
| ?- List_Append(A,[E,I,O,U],Newlist).
```

```
Newlist = [A,E,I,O,U]
```

```
Yes
```

```
| ?- List_Append(E,[E,I,O,U],Newlist).
```

```
Newlist = [E,I,O,U]
```

```
Yes
```

```
| ?- List_Append([A,B],[E,I,O,U],Newlist).
```

```
Newlist = [[A,B],E,I,O,U]
```

```
Yes
```

```
| ?-
```

CHAPTER 6

Objective:

Write A Prolog Code To Find The Maximum Element Of A List.

Requirements:

Processors: Intel Atom® processor or Intel® Core™ i3 processor., Disk space: 1 GB. ,

Operating systems: Windows* 7 or later, macOS, and Linux. Software Prolog.

Theory:

This Operation Is Used To Find The Maximum Element From A List. We Will Define A Predicate, List_Max_Elem(List, Max), Then This Will Find Max Element From The List And Return.

1. If There Is Only One Element, Then It Will Be The Max Element.
2. Divide The List As [X,Y|Tail]. Now Recursively Find Max Of [Y|Tail] And Store It Into Maxrest, And Store Maximum Of X And Maxrest, Then Store It To Max.

Program:

```
Max_Of_Two(X,Y,X) :- X >= Y.
```

```
Max_Of_Two(X,Y,Y) :- X < Y.
```

```
List_Max_Elem([X],X).
```

```
List_Max_Elem([X,Y|Rest],Max) :-
```

```
    List_Max_Elem([Y|Rest],Maxrest),
```

```
    Max_Of_Two(X,Maxrest,Max).
```

Output:

```
| ?- List_Max_Elem([8,5,3,4,7,9,6,1],Max).
```

Max = 9 ?

Yes

```
| ?- List_Max_Elem([5,12,69,112,48,4],Max).
```

Max = 112 ?

Yes

```
| ?-
```

CHAPTER 7

Objective:

Write A Prolog Predicate To Reverse The Order Of The Elements Of A Given Input List.

Requirements:

Processors: Intel Atom® processor or Intel® Core™ i3 processor., Disk space: 1 GB. ,

Operating systems: Windows* 7 or later, macOS, and Linux. Software Prolog.

Theory:

Suppose We Have A List $L = [A, B, C, D, E]$, And We Want To Reverse The Elements, So The Output Will Be $[E, D, C, B, A]$. To Do This, We Will Create A Clause, `List_Reverse(List, Reversedlist)`. Following Are Some Observations –

1. If The List Is Empty, Then The Resultant List Will Also Be Empty.
2. Otherwise Put The List Items Namely, `[Head|Tail]`, And Reverse The Tail Items Recursively, And Concatenate With The Head.
3. Otherwise Put The List Items Namely, `[Head|Tail]`, And Reverse The Tail Items Recursively, And Concatenate With The Head.

Program:

`List_Concat([], L, L).`

`List_Concat([X1|L1], L2, [X1|L3]) :- List_Concat(L1, L2, L3).`

`List_Rev([], []).`

`List_Rev([Head|Tail], Reversed) :-`

`List_Rev(Tail, Revtail), List_Concat(Revtail, [Head], Reversed).`

Output:

`| ?- List_Rev([A,B,C,D,E], Newlist).`

`Newlist = [E,D,C,B,A]`

Yes

`| ?- List_Rev([A,B,C,D,E], [E,D,C,B,A]).`

Yes

`| ?-`